



Desarrollo Web

FACULTAD CIENCIAS DE LA INGENIERÍA, CARRERA SOFTWARE -
PRESENCIAL

Unidad 1

LENGUAJE DEL LADO DEL CLIENTE - FRONT END

Tema 2

HTML

MSc. Josué Guevara
rguevarar@unemi.edu.ec

Subtemas

Subtema 1: Estructura global.

Subtema 2: Semántica.

Subtema 3: Video y Audio.

Subtema 4: Practica.

Objetivo

Conocer la estructura de trabajo de las páginas web, su diseño, sus etiquetas base y elementos principales.

Introducción HTML

HTML (HiperText Markup Language), es la piedra angular de la web, permitiendo la creación de páginas y aplicaciones accesibles en línea. La estructura global de un documento HTML define su organización y el orden que se presentan los elementos. La semántica de HTML es crucial, ya que el uso adecuado de etiquetas y elementos no solo facilita la accesibilidad, sino también la optimización para motores de búsqueda. Con la llegada de HTML5, se han introducido nuevas capacidades, como el soporte nativo para video y audio.

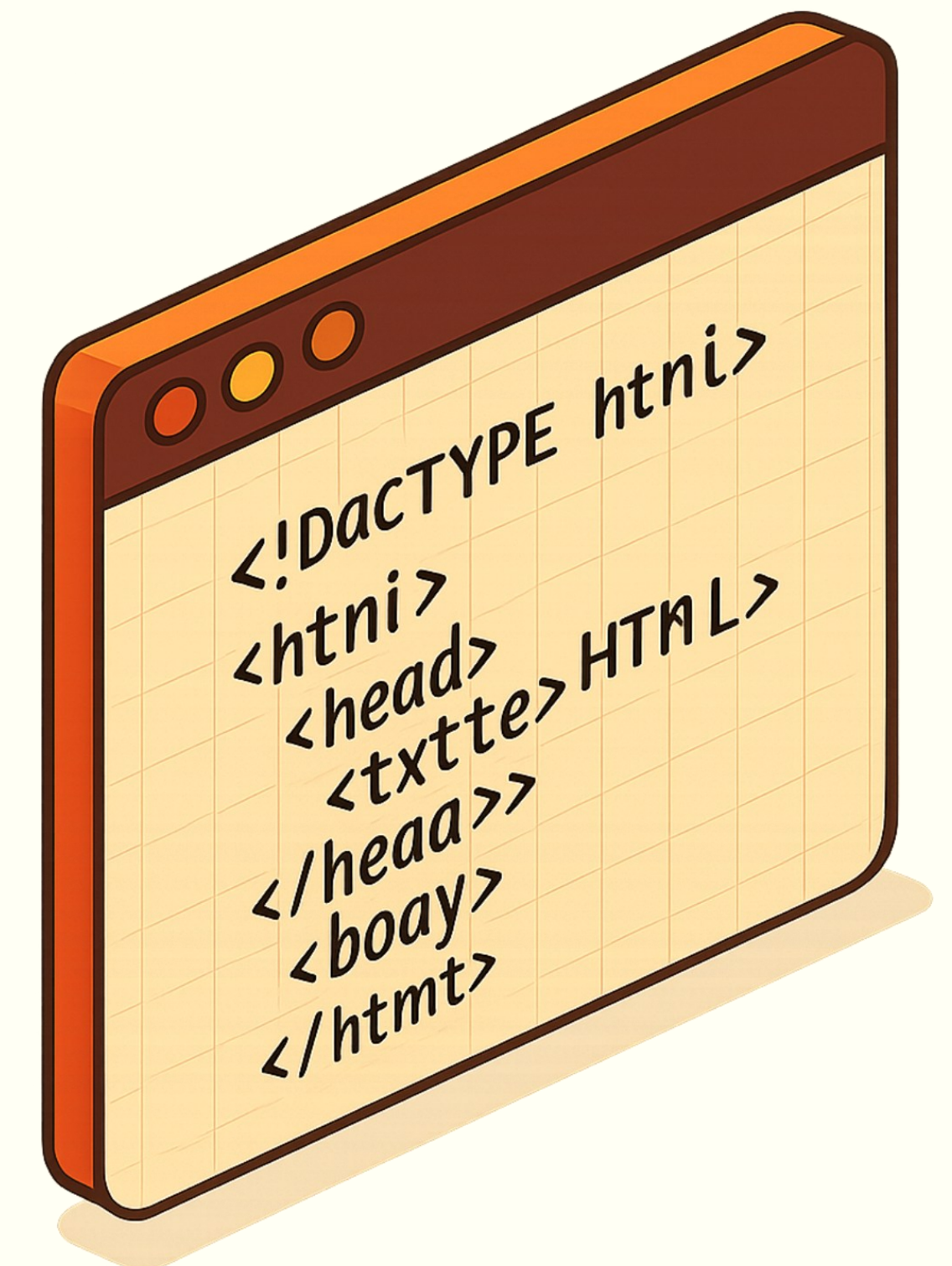
¿Qué recibe realmente el navegador?

Cuando el servidor responde a una petición HTTP, devuelve un archivo de texto plano. Pero... ¿cómo sabe el navegador qué es un título y qué es un párrafo?

```
HTTP/1.1 200 OK
Content-Type: text/html

<h1>Catálogo</h1>
<p>Productos disponibles</p>
```

❓ ¿Cómo distingue el navegador el significado de cada parte del contenido?



$\vdash \perp \sim \exists x (\neg \forall y (x = y) \wedge \forall z (x = z))$

Lenguaje de marcado que describe la **estructura y el significado** del contenido web.

HTML **NO** es un lenguaje de programación. No tiene lógica, condiciones ni bucles.

Esqueleto / estructura

Apariencia / diseño

Comportamiento / lógica

Anatomía de un elemento HTML

Cada elemento tiene partes bien definidas. Analicemos un enlace real del catálogo:

```
<a href="productos.html">Ver productos</a>
```

1

Etiqueta de apertura

`<a>` — indica el inicio del elemento

2

Atributo + valor

`href="productos.html"` — configura el comportamiento

3

Contenido

`Ver productos` — lo que ve el usuario

4

Etiqueta de cierre

`</a>` — indica el fin del elemento

Elementos y atributos

El elemento dice *qué es*

Define el tipo de contenido que representa.

```

```

```
<a href="/carrito"  
target="_blank">Ver carrito</a>
```

Los atributos dicen *cómo*

src

Ruta o URL del recurso
(imagen, audio...)

alt

Texto alternativo para
accesibilidad

href

Destino de un enlace

target

Dónde se abre el enlace
(`_blank`, `_self`)

Estructura mínima de un documento HTML

```
<!DOCTYPE html>
<html lang="es">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport"
      content="width=device-width">
    <title>Catálogo de Productos</title>
  </head>
  <body>
    <h1>Catálogo</h1>
    <p>Productos disponibles</p>
  </body>
</html>
```

→ `<!DOCTYPE html>`

Declara la versión de HTML (HTML5)

→ `<html lang="es">`

Raíz del documento; idioma para lectores de pantalla

→ `<head>`

Metadatos, configuración, enlaces a recursos

→ `<body>`

Todo el contenido visible para el usuario

HEAD vs BODY



🧠 **<head> —**
Configuración del
proyecto

- **<title> —** Título en la pestaña
- **<meta charset> —** Codificación de caracteres
- **<meta viewport> —** Adaptación a móviles
- **<link> —** Enlace a archivos CSS

El usuario no lo ve, pero el navegador lo necesita.

👁️ **<body> —** Lo que experimenta el usuario

- Títulos y párrafos
- Imágenes y vídeos
- Formularios y botones
- Navegación y secciones

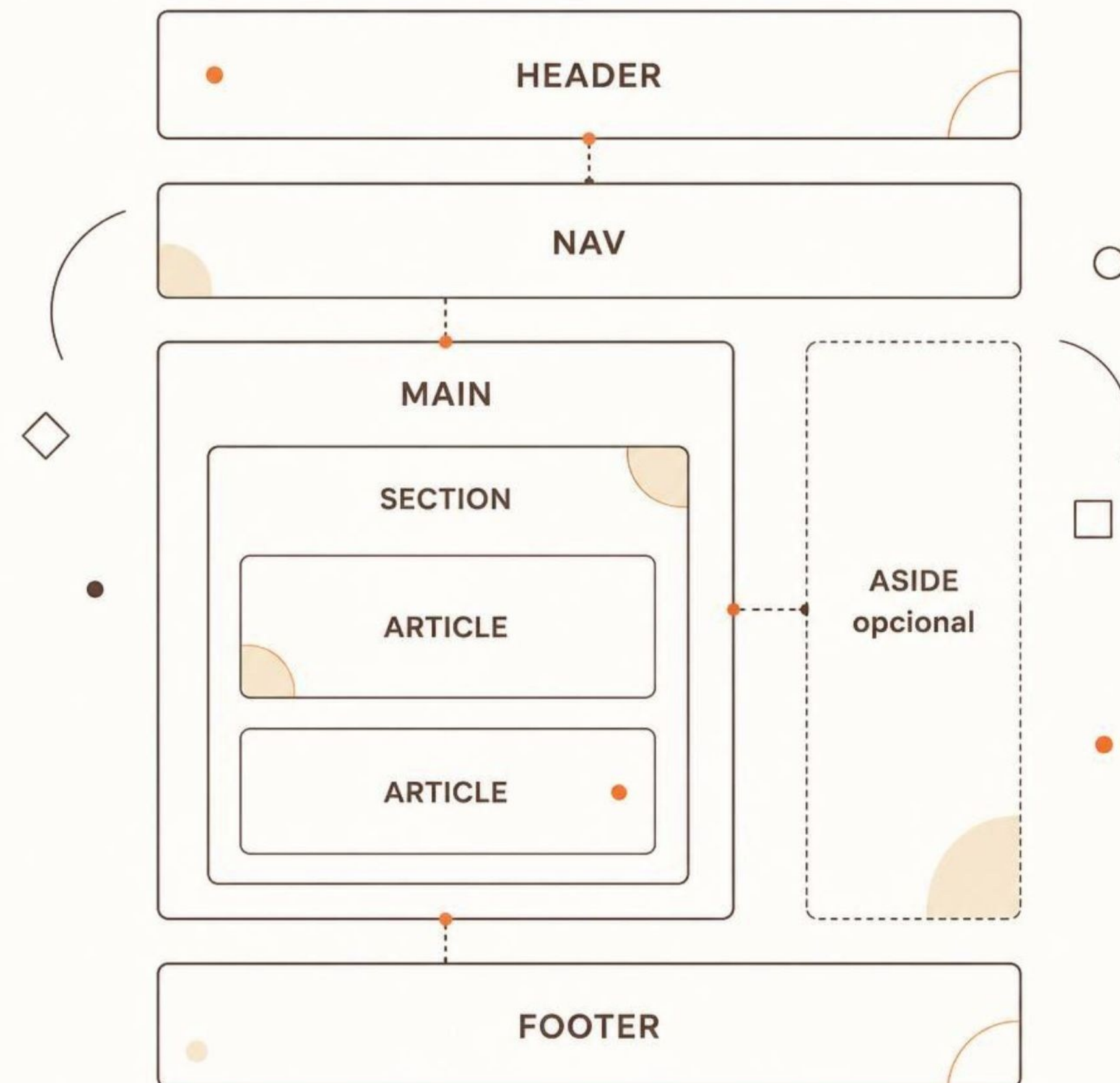
Todo lo visible e interactivo vive aquí.

Estructura semántica HTML5

Etiqueta	Uso correcto	Cuándo NO usarla
<code><div></code>	Contenedor genérico sin significado semántico	Nunca como sustituto de etiquetas semánticas
<code><section></code>	Agrupar contenido temáticamente relacionado	Si el contenido es independiente (usar <code><article></code>)
<code><article></code>	Contenido autónomo y redistribuible	Para agrupaciones sin contexto propio
<code><header></code> / <code><footer></code>	Encabezado o pie de sección o de página	Como contenedores decorativos
<code><main></code>	Contenido principal único de la página	Nunca debe repetirse en el mismo documento

HTML5 introdujo un conjunto de etiquetas semánticas que reemplazaron el uso indiscriminado de `<div>`. Cada elemento tiene un propósito específico que comunica significado, no solo presentación visual.

Estructura semántica de una página



<header>

Encabezado: logo, título y acciones globales

<nav>

Menú de navegación principal

<main>

Contenido principal de la página (único)

<section>

Agrupación temática dentro de main

<article>

Contenido autónomo y reutilizable

<aside>

Contenido complementario o lateral

<footer>

Pie: contacto, copyright, enlaces legales

Impacto real del HTML semántico

En sistemas modernos, HTML cumple múltiples funciones simultáneas que van mucho más allá de «pintar» una pantalla:

- Es la base sobre la que se construye el **DOM** (Document Object Model)
- Es interpretado y renderizado directamente por el navegador
- Es consumido y transformado por frameworks como **React** o **Vue**
- Impacta directamente en **SEO, accesibilidad** y **rendimiento percibido**

SEO

Google interpreta la estructura semántica para indexar el contenido con mayor precisión.

Accesibilidad

Los lectores de pantalla dependen de etiquetas semánticas correctas para navegar.

Frameworks

React y Vue generan su virtual DOM a partir del HTML real del documento.

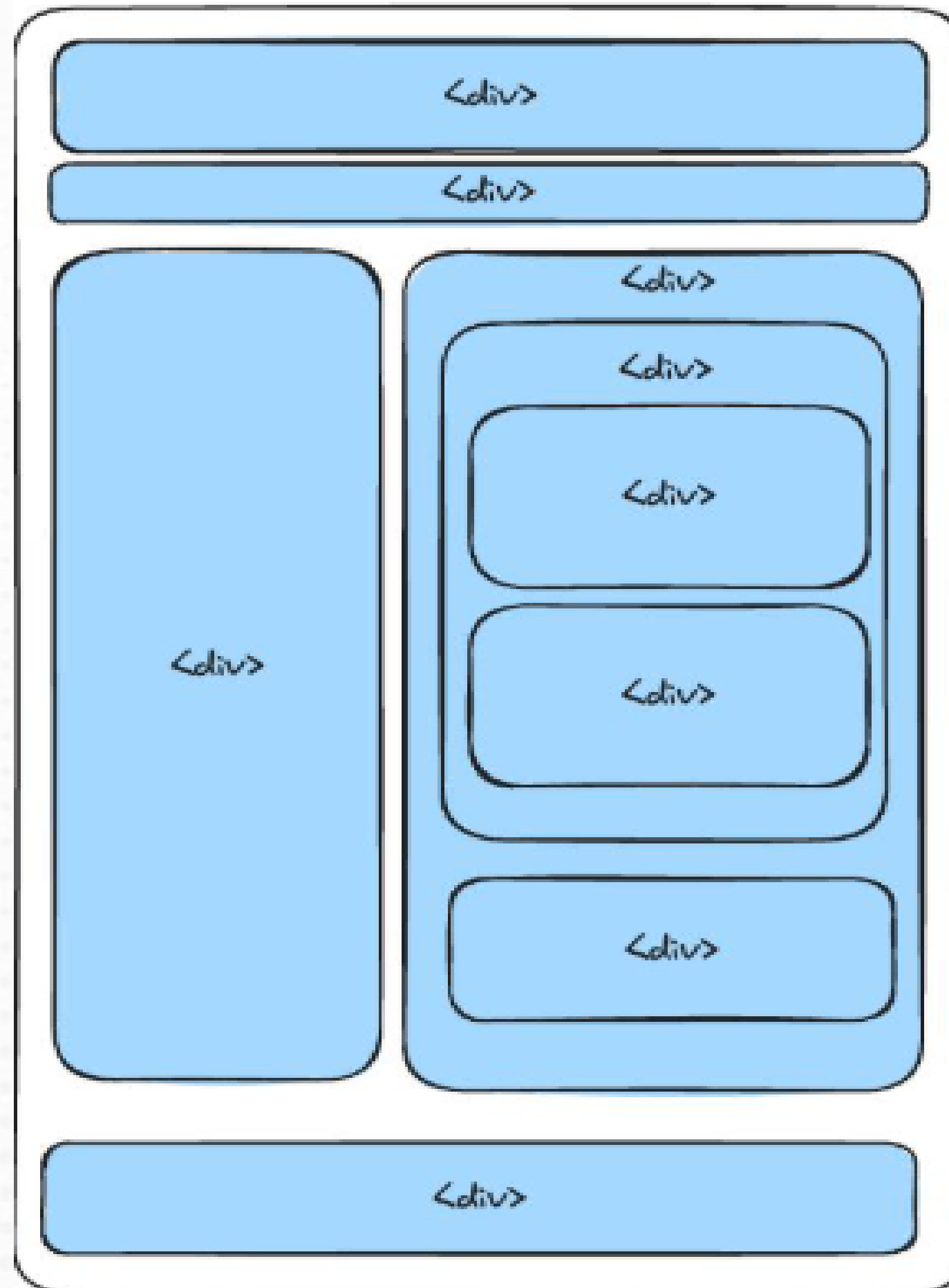
HTML Básico vs. HTML Semántico

La diferencia entre una estructura con `<div>` genéricos y una estructura semántica es profunda: no solo afecta a la legibilidad del código, sino al comportamiento del navegador, los motores de búsqueda y las herramientas de accesibilidad.

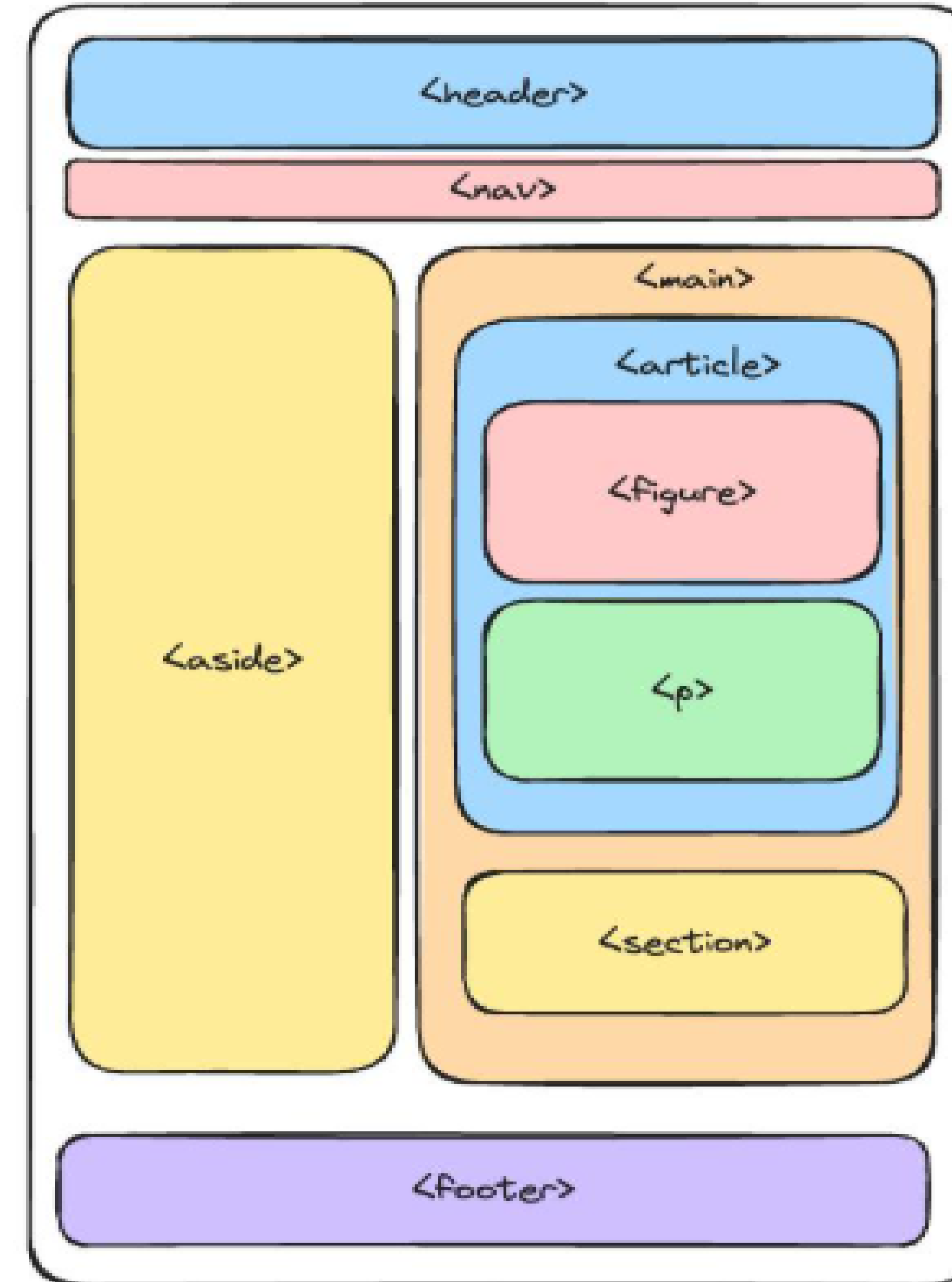
 **Error habitual en perfiles junior:** usar exclusivamente `<div>` para estructurar el documento. Esto produce un código sin significado semántico, difícil de mantener y perjudicial para el SEO y la accesibilidad.

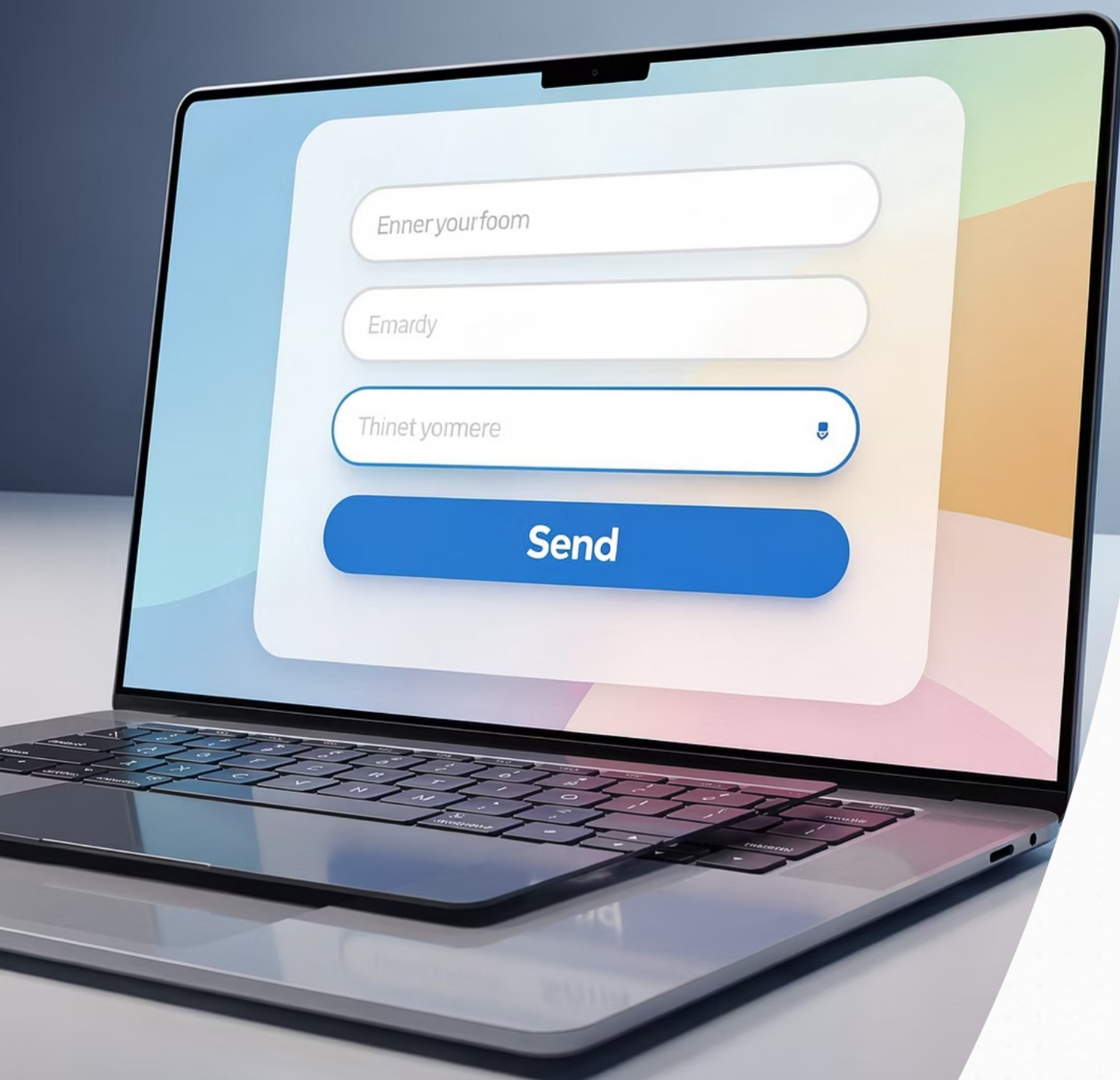
HTML Básico vs. HTML Semántico

Estructura HTML Básica



Estructura HTML Semántica





Formularios HTML: el núcleo de la interacción

Los formularios son el principal mecanismo mediante el cual los usuarios envían datos a una aplicación web. Un formulario mal construido compromete la seguridad, la experiencia de usuario y la integridad de los datos.

¿Por qué los formularios son críticos?



Puente frontend-backend

Los formularios son el canal principal por el que los datos viajan desde el navegador hasta el servidor. Su estructura condiciona la arquitectura de la API que los consume.



Impacto directo en UX

Un formulario mal etiquetado, sin validaciones o con retroalimentación deficiente aumenta la tasa de abandono y deteriora la percepción del producto.



Primera línea de validación

La validación HTML5 nativa reduce la carga de procesamiento en el backend y proporciona feedback inmediato, antes de que el dato llegue al servidor.

Validación visual y feedback de errores

Buenas prácticas de validación

El feedback de error debe ser **específico**, aparecer **en contexto** (junto al campo afectado) y no bloquearse hasta el envío. HTML5 proporciona mecanismos nativos para esto:

- El atributo `required` marca el campo como obligatorio
- `pattern` permite validar contra expresiones regulares
- `minlength` / `maxlength` controlan la longitud del valor
- El pseudo-selector `:invalid` en CSS permite estilizar campos erróneos



La validación HTML5 no sustituye a la validación en backend. Ambas capas son necesarias: la del cliente mejora la UX, la del servidor garantiza la seguridad.

Tipos de `input` en HTML

El atributo `type` no es meramente decorativo: define el comportamiento del teclado en móvil, activa validaciones nativas del navegador y comunica semántica a los lectores de pantalla. Elegir el tipo correcto es una decisión de accesibilidad, no solo de UX.

Tipo

`text`

`email`

`password`

`number`

`date`

`checkbox`

`radio`

`file`

Uso principal

Texto libre genérico

Direcciones de correo

Contraseñas

Datos numéricos

Selección de fechas

Selección múltiple

Selección única en grupo

Subida de archivos

Ventaja técnica

Base de todos los inputs de texto

Validación automática de formato

Oculto visualmente la entrada

Teclado numérico en móvil, min/max

Selector nativo sin JavaScript

Envía array de valores

Agrupación por atributo `name`

Filtrado por tipo con `accept`

Estructura de formulario profesional

```
<form action="/registro" method="POST">
  <label for="email">Correo electrónico</label>
  <input
    type="email"
    id="email"
    name="email"
    required
    autocomplete="email">
  <label for="password">Contraseña</label>
  <input
    type="password"
    id="password"
    name="password"
    required
    minlength="8">
  <button type="submit">Registrarse</button>
</form>
```

Aspectos clave del código

→ **action y method**

Definen el endpoint y el verbo HTTP. Usar **POST** para datos sensibles, nunca **GET**.

→ **Vinculación for/id**

El **label** debe referenciarse con **for** al **id** del **input**. Crítico para accesibilidad.

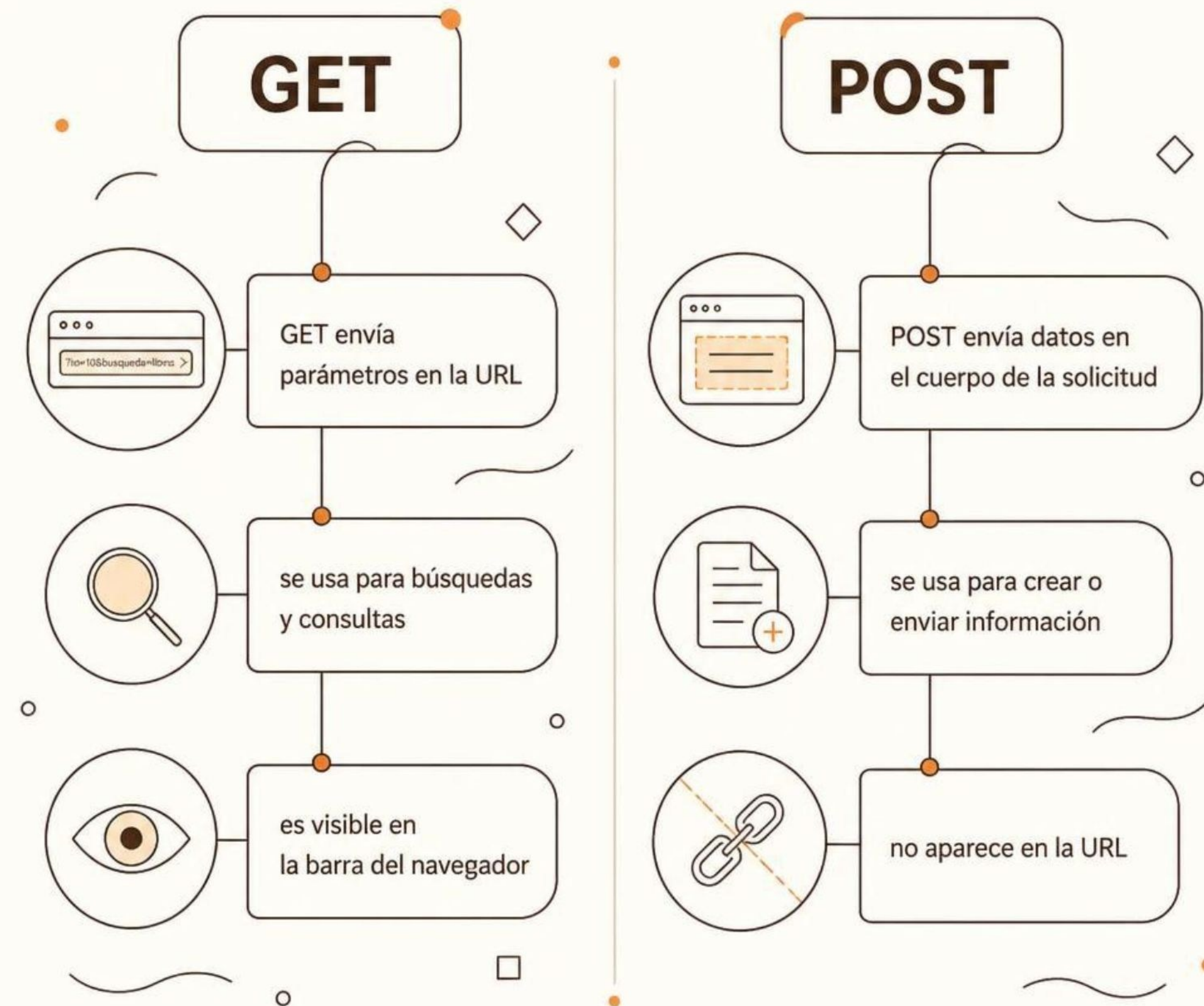
→ **autocomplete**

Facilita el autocompletado del navegador y mejora la experiencia en formularios de autenticación.

→ **minlength**

Validación nativa de longitud mínima sin necesidad de JavaScript adicional.

GET vs POST en formularios



GET — búsqueda del catálogo

`/productos?categoria=ropa&talla=M`

Los parámetros son visibles en la URL.
Ideal para búsquedas que puedan compartirse o guardarse.

POST — registro de usuario

Los datos viajan en el **cuerpo** de la solicitud, no en la URL. Ideal para crear recursos o enviar información sensible.



POST no cifra los datos. La confidencialidad depende de **HTTPS**, no del método HTTP.

BIBLIOGRAFÍA

- Mastermarketing-valencia.com. (2019). ¿Cuándo nació Internet? Historia y evolución. Recuperado de <https://n9.cl/hlnxf>
- Mastermarketing-valencia.com. (2019). Infografía-La web Historia. Recuperado de <https://n9.cl/u7bct>
- Significado de Web. (2021). En Significados. Recuperado de <https://n9.cl/5gpv4>
- Schiaffarino, A. (2019). Modelo cliente servidor. Infranetworking. Recuperado de <https://n9.cl/bee5>
- ARQUITECTURA CLIENTE-SERVIDOR. (2021). Nota Tecnológica. Recuperado de <https://n9.cl/kc28a>
- Berners-Lee, T. (2000). Weaving the web: The original design and ultimate destiny of the World Wide Web. Harper Business.
- Tanenbaum, A. S., & van Steen, M. (2001). Distributed systems: Principles and paradigms (1st ed.). Pearson.
- Robbins, J. N. (2012). Learning Web Design: A Beginner's Guide to HTML, CSS, JavaScript, and Web Graphics (4th ed.). O'Reilly Media.

UNEMI
UNIVERSIDAD ESTATAL DE MILAGRO